

# Dynamic Programming

## Day 1

- Priyansh Agarwal

Why Dynamic Programming?

**“THOSE WHO CANNOT  
REMEMBER THE PAST  
ARE CONDEMNED  
TO REPEAT IT”**

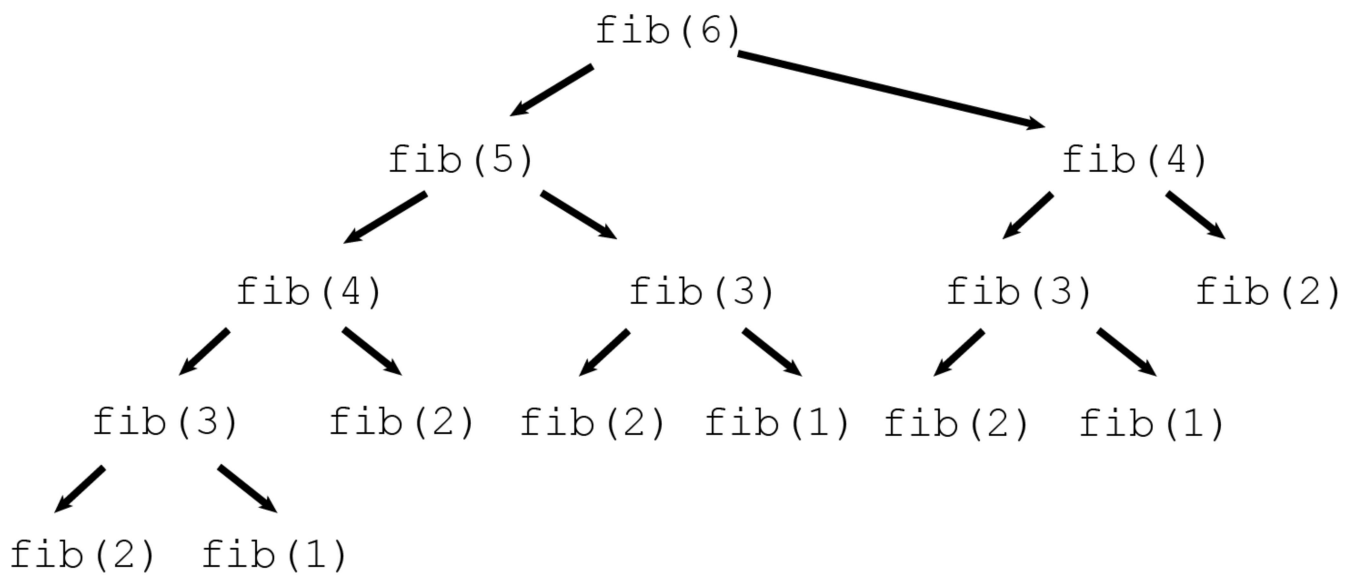
*-George Santayana-*

# Why Dynamic Programming?

- Overlapping subproblems
- Maximize/Minimize some value
- Finding number of ways
- Covering all cases (DP vs Greedy)
- Check for possibility

## Need of DP

- Let's understand this from a problem
  - Find  $n^{\text{th}}$  fibonacci number
  - $F(n) = F(n - 1) + F(n - 2)$
  - $F(1) = F(2) = 1$



Any problem here?



Overlapping subproblems?

# Memoization

- Why calculate  $F(x)$  again and again when we can calculate it once and use it every time it is required?
  - Check if  $F(x)$  has been calculated
    - If No, calculate it and store it somewhere
    - If Yes, return the value without calculating again

## Without DP

```
int functionEntered = 0;
int helper(int n){
    functionEntered++;
    if(n == 1 || n == 2){
        return 1;
    }
    return helper(n - 1) + helper(n - 2);
}
void solve(){
    int n;
    cin >> n;
    cout << helper(n) << endl;
    cout << functionEntered << endl;
}
```

functionEntered = 1664079  
with n = 30

## With DP

```
int functionEntered = 0;
int dp[40];
int helper(int n){
    functionEntered++;
    if(n == 1 || n == 2){
        return 1;
    }
    if(dp[n] != -1)
        return dp[n];
    return dp[n] = helper(n - 1) + helper(n - 2);
}
void solve(){
    int n;
    cin >> n;
    for(int i = 0; i <= n; i++)
        dp[i] = -1;
    cout << helper(n) << nline;
    cout << functionEntered << nline;
}
```

functionEntered = 57  
with n = 30

## Let's solve another problem!

Given a 2D grid (N X M) with numbers written in each cell, find the path from top left (0, 0) to bottom right (n - 1, m - 1) with minimum sum of values on the path

|   |   |   |
|---|---|---|
| 1 | 5 | 8 |
| 6 | 2 | 7 |
| 9 | 3 | 4 |

## Naive Way

Explore all paths. Standing at  $(i, j)$  try both possibilities  $(i + 1, j)$ ,  $(i, j + 1)$

Every cell has two choices

Time complexity:  $O(2^{m*n})$ ?

Actual Time complexity:  $O(C(n + m - 2, m - 1))$

## Efficient Way

Overlapping subproblems

Memoization

Time complexity:  $O(n * m)$

Space complexity:  $O(n * m)$

```

int grid[n][m]; // input matrix
int dp[n][m]; // every value here is -1
// subproblem: f(i, j) represents minimum sum path from (i, j) to (n - 1, m - 1)
int f(int i, int j){ //
    if(i >= n || j >= m){ // moving outside the grid // not allowed
        return INF;
    }
    if(i == n - 1 && j == m - 1) // reached the destination
        return grid[n - 1][m - 1];

    if(dp[i][j] != -1) // this state has been calculated before
        return dp[i][j];

    // state never calculated before
    dp[i][j] = grid[i][j] + min(f(i, j + 1), f(i + 1, j));
    return dp[i][j];
}

void solve(){
    cout << f(0, 0) << endl;
}

```

## Important Terminology

**State:** A subproblem that we want to solve. The subproblem may be complex or easy to solve but the final aim is to solve the final problem which may be defined by a relation between the smaller subproblems. Represented with some parameters.

**Transition:** Calculating the answer for a state (subproblem) by using the answers of other smaller states (subproblems). Represented as a relation b/w states.

## Exercise

Fibonacci Problem:

- State
  - $dp[i]$  or  $f(i)$  meaning  $i^{\text{th}}$  fibonacci number
- Transition
  - $dp[i] = dp[i - 1] + dp[i - 2]$

## Exercise

Matrix Problem:

- State
  - $dp[i][j]$  = shortest sum path from  $(i, j)$  to  $(n - 1, m - 1)$
- Transition
  - $dp[i][j] = grid[i][j] + \min(dp[i + 1][j], dp[i][j + 1])$



## Let's solve another problem

Given an array of integers (both positive and negative). Pick a subsequence of elements from it such that no 2 adjacent elements are picked and the sum of picked elements is maximized.

|   |   |   |     |    |   |
|---|---|---|-----|----|---|
| 1 | 4 | 2 | -10 | 10 | 5 |
|---|---|---|-----|----|---|

Sum = 14

|   |   |   |     |    |   |
|---|---|---|-----|----|---|
| 1 | 4 | 2 | -10 | 10 | 5 |
|---|---|---|-----|----|---|

Sum = 13

## Some ways to solve the problem

### 1. Having 2 parameters to represent the state

State:

$dp[i][0]$  = maximum sum in (0 to i) if we don't pick  $i^{th}$  element

$dp[i][1]$  = maximum sum in (0 to i) if we pick  $i^{th}$  element

Transition:

$dp[i][0] = \max(dp[i - 1][1], dp[i - 1][0])$

$dp[i][1] = arr[i] + dp[i - 1][0]$

Final Answer:

$\max(dp[n - 1][0], dp[n - 1][1])$

# Some ways to solve the problem

## 2. Having only 1 parameter to represent the state

State:

$dp[i]$  = max sum in (0 to i) not caring if we picked  $i^{\text{th}}$  element or not

Transition: 2 cases

- pick  $i^{\text{th}}$  element: cannot pick the last element :  $arr[i] + dp[i - 2]$

- leave  $i^{\text{th}}$  element: can pick the last element :  $dp[i - 1]$

$dp[i] = \max(arr[i] + dp[i - 2], dp[i - 1])$

Final Answer:

$dp[n - 1]$

```
int a[n]; // input array
int dp[n]; // filled with -INF to represent uncalculated state
// f(i) = max sum till index i
int f(int index){
    if(index < 0) // reached outside the array
        return 0;
    if(dp[index] != -INF) // state already calculated
        return dp[index];

    // try both cases and store the answer
    dp[index] = max(a[index] + f(index - 2), f(index - 1));
    return dp[index];
}

void solve(){
    cout << f(n - 1) << endl;
}
```

# Time and Space Complexity in DP

Time Complexity:

Estimate:  $\text{Number of States} * \text{Transition time for each state}$

Exact:  $\text{Total transition time for all states}$

Space Complexity:

$\text{Number of States} * \text{Space required for each state}$

## Dynamic Programming Day 2

- Priyansh Agarwal

# Solving Homework Problems

## Problem 1: [Link](#)

- State:
  - $dp[i]$  = number of ways to get sum ==  $i$
- Transition:
  - $dp[i] = dp[i - 1] + dp[i - 2] + \dots + dp[i - 6]$
- Final Subproblem:
  - $dp[n]$

## Problem 2: [Link](#)

- State:
  - $dp[k] = \text{min coins required to make sum} == k$
- Transition:
  - $dp[k] = 1 + \min\{dp[k - \text{coins}_i]\} \quad (0 \leq i \leq n - 1)$
- Final Subproblem:
  - $dp[x]$

## Problem 3: [Link](#)

- State:
  - $dp[i] = \text{number of ways to make sum} == i$
- Transition:
  - $dp[i] = \text{sum of } dp[i - \text{coins}_j] \quad (0 \leq j \leq n - 1)$
- Final Subproblem:
  - $dp[x]$

# Recursive vs Iterative DP (in Day 2)

| Recursive                            | Iterative                                                                                                  |
|--------------------------------------|------------------------------------------------------------------------------------------------------------|
| Slower (runtime)                     | Faster (runtime)                                                                                           |
| No need to care about the flow       | Important to calculate states in a way that current state can be derived from previously calculated states |
| Does not evaluate unnecessary states | All states are evaluated                                                                                   |
| Cannot apply many optimizations      | Can apply optimizations                                                                                    |

## General Technique to solve any DP problem

### 1. State

Clearly define the subproblem. Clearly understand when you are saying  $dp[i][j][k]$ , what does it represent exactly

### 2. Transition:

Define a relation b/w states. Assume that states on the right side of the equation have been calculated. Don't worry about them.

### 3. Base Case

When does your transition fail? Call them base cases answer before hand. Basically handle them separately.

### 4. Final Subproblem

What is the problem demanding you to find?

# Solving Classical Problems

## Problem 1: [Link](#)

- State:
  - $dp[x]$  = min steps to convert  $x$  to 0
- Transition:
  - $dp[x] = \min(dp[x - \text{some digit of } x]) + 1$
- Base Case:
  - $dp[0] = 0$
- Final Subproblem:
  - $dp[n]$

## Problem 2: [Link](#)

- State:
  - $dp[i][j]$  = number of ways to go from  $(0, 0)$  to  $(i, j)$
- Transition:
  - $dp[i][j] = dp[i - 1][j] + dp[i][j - 1]$
- Base Case:
  - step1:  $dp[0][0] = 1$ , step2:  $dp[i][j] = 0$ , when  $(i, j)$  is a trap
- Final Subproblem:
  - $dp[n - 1][n - 1]$

## Problem 2: [Link](#)

- State:
  - $dp[i][j]$  = number of ways to go from  $(i, j)$  to  $(n - 1, n - 1)$
- Transition:
  - $dp[i][j] = dp[i + 1][j] + dp[i][j + 1]$
- Base Case:
  - $dp[n - 1][n - 1] = 1$ ,  $dp[i][j] = 0$ , when  $(i, j)$  is a trap
- Final Subproblem:
  - $dp[0][0]$



# Space Optimization

- What other state does our current state depend on?
- Do we need answers to all subproblems at all times?
- Well, let's store only relevant states then!
- But wait, does this always work?
  - What if the final subproblem requires all the states?
  - What if we need to backtrack? [more on this in later lectures]

- Fibonacci Problem
  - $dp[i]$  depends on  $dp[i - 1]$ ,  $dp[i - 2]$
- Grid Problem
  - $dp[i][j]$  depends on  $dp[i + 1][j]$ ,  $dp[i][j + 1]$
- Dice Problem in Homework
  - $dp[i]$  depends on  $dp[i - k]$  ( $1 \leq k \leq 6$ )

# Dynamic Programming

## Day 3

- Priyansh Agarwal

## Solving Homework Problems

## Problem 1: [Link](#)

- State:
  - $dp[k][i] \Rightarrow$  current sum =  $k$ , standing at  $i^{\text{th}}$  coin, number of ways to make sum =  $X$
- Transition:
  - $dp[k][i] = dp[k + \text{coin}_i][i] + dp[k][i + 1]$
- Base Case:
  - $dp[X][i] = 1$ ,  $dp[> X][i] = 0$ ,  $dp[Y][n] = 0$
- Final Subproblem:
  - $dp[0][0]$

## Problem 2: [Link](#)

- State:
  - $dp[i][j] =$  max pages you can read when allowed to read the first  $i$  books and allowed cost is  $j$
- Transition:
  - $dp[i][j] = \max(dp[i - 1][j], \text{pages}[i - 1] + dp[i - 1][j - \text{cost}[i - 1]])$
- Base Case:
  - $dp[0][\text{anything}] = 0$
- Final Subproblem:
  - $dp[n][x]$

## Problem 3: [Link](#)

- State:
  - $dp[i][x]$  = number of ways to fill  $[0..i]$  s.t the  $i$ th element is  $x$
- Transition:
  - $dp[i][x] = dp[i - 1][x - 1] + dp[i - 1][x] + dp[i - 1][x + 1]$
- Base Case:
  - $dp[0][x] = (arr[0] == 0 \mid \mid arr[0] == x ? 1 : 0)$
- Final Subproblem:
  - $dp[n - 1][1] + dp[n - 1][2] + \dots + dp[n - 1][m]$

## State Optimization

- Ask yourself do you need all the parameters in the dp state?
- If you have  $dp[a][b][c]$ , and  $a + b = c$ , do you need to store  $c$  as a parameter or can you just compute it on spot?
- If you can compute a parameter in dp state from other parameters, no need to store it.
- Which parameters should you remove? Highest

# Transition Optimization

- Observe the transition equation.
- Can you do some pre-computation to evaluate the equation faster?
- Using clever observations.
- Using range query data structures

Solving More Classical Problems

## Problem 1: [Link](#)

- State:
  - $dp[i][0]$  = number of ways to fill rows from (i to n) such that last row had 2 growing blocks
  - $dp[i][1]$  = number of ways to fill rows from (i to n) such that last row had a  $(1 * 2)$  type of growing block

## Problem 1: [Link](#)

- Transition:
  - for  $dp[i][0]$ 
    - pos1: close both blocks  $\rightarrow dp[i + 1][0] + dp[i + 1][1]$ 
      - pos11: start 2 blocks of  $1 * 1$  each  $= dp[i + 1][0]$
      - pos12: start 1 block of  $1 * 2 = dp[i + 1][1]$
    - pos2: don't close any of them  $\rightarrow dp[i + 1][0]$
    - pos3 and 4: close one of them  $\rightarrow 2 * dp[i + 1][0]$
  - $dp[i][0] = pos1 + pos2 + pos3 + pos4 =$ 
    - $= dp[i + 1][0] + dp[i + 1][1] + dp[i + 1][0] + 2 * dp[i + 1][0]$
    - $= 4 * dp[i + 1][0] + dp[i + 1][1]$

## Problem 1: Link

- Transition:

- for  $dp[i][1]$ 
  - pos1: close the complete block:
    - pos11: start 2 blocks of  $1 * 1$  each =  $dp[i + 1][0]$
    - pos12: start 1 block of  $1 * 2$  =  $dp[i + 1][1]$
  - pos2: don't close the block ->  $dp[i + 1][1]$
- $dp[i][1] = pos1 + pos2$ 
  - =  $dp[i + 1][0] + dp[i + 1][1] + dp[i + 1][1]$
  - =  $2 * dp[i + 1][1] + dp[i + 1][0]$

## Problem 1: Link

- Base case:

- $dp[n - 1][0] = 1$
- $dp[n - 1][1] = 1$

- Final subproblem:

- $dp[0][1] + dp[0][0]$



# Dynamic Programming Day 4

- Priyansh Agarwal



Solving Homework Problems



## Problem 1: [Link](#)

- State:
  - $dp[i][j]$  = edit distance b/w string  $A[i:n]$  and  $B[j:m]$
- Transition:
  - $dp[i][j] = (a[i] == b[j]) ? dp[i + 1][j + 1] : 1 + \min(dp[i][j + 1], dp[i + 1][j], dp[i + 1][j + 1])$
- Base Case:
  - $dp[n][j] = m - j, dp[i][m] = n - i$
- Final Subproblem:
  - $dp[0][0]$

## Problem 2: [Link](#)

- State:
  - $dp[i][j]$  = min cuts to make rectangle of  $(i * j)$  into squares
- Transition:
  - $dp[i][j] = \min(1 + \min(dp[i][k] + dp[i][j - k]), 1 + \min(dp[k][j] + dp[i - k][j]))$
- Base Case:
  - $dp[i][i] = 0$
- Final Subproblem:
  - $dp[n][m]$

## Problem 3: [Link](#)

- State:
  - $dp[i]$  = set of sums possible with coins from 1 to  $i^{\text{th}}$  coin
- Transition:
  - $dp[i] \Rightarrow$  add or not add the current coin to all the sums possible in the previous state
- Base Case:
  - $dp[0] = \{0\}$
- Final Subproblem:
  - $dp[n]$

## Problem 3: [Link](#)

```
void solve(){
    vector<bool> possible(1e5 + 1, false);
    possible[0] = true;
    for(int i = 1; i <= n; i++){
        vector<bool> current(1e5 + 1, false);
        int currentDenomination = coins[i - 1];
        for(int j = 0; j <= 1e5; j++){
            if(possible[j]){
                current[j + currentDenomination] = true;
                current[j] = true;
            }
        }
        possible = current;
    }
    for(int i = 1; i <= 1e5; i++){
        if(possible[i])
            cout << i << " ";
    }
    cout << endl;
}
```

# Answer Construction

- Grid problem: Find the actual path with the minimum sum.
- Minimizing coins problem: Find the actual choice of coins.
- At every state we are making some optimal choice.
  - If we store this choice, we can be sure that if we are at any state we know what is the best choice.
  - Start from the state that contains your final subproblem and keep making the best choice (which was already stored) until you reach the end.

## Answer Construction - Grid Problem

```
int n = 3, m = 3;
vector<vector<int>> grid(3, vector<int>(3));
vector<vector<pair<int, int>>> dp(n, vector<pair<int, int>>(m, {-1, 0}));
// 0 -> take a down direction
// 1 -> take a right direction
int f(int i, int j){
    if(i == n || j == m)
        return 1e9;
    if(i == n - 1 && j == m - 1)
        return grid[n - 1][m - 1];
    if(dp[i][j].first != -1)
        return dp[i][j].first;

    int ans1 = f(i + 1, j);
    int ans2 = f(i, j + 1);
    if(ans1 < ans2){
        dp[i][j].second = 0;
    }else{
        dp[i][j].second = 1;
    }
    return dp[i][j].first = grid[i][j] + min(ans1, ans2);
}
```

# Answer Construction - Grid Problem

```
void solve(){
    for(int i = 0; i < 3; i++){
        for(int j = 0; j < 3; j++){
            cin >> grid[i][j];
        }
    }
    cout << f(0, 0) << endl;
    pair<int, int> current = {0, 0};
    while(current != mp(n - 1, m - 1)){
        cout << current.first << " " << current.second << endl;
        if(dp[current.first][current.second].second == 0)
            current.first++;
        else
            current.second++;
    }
    cout << current.first << " " << current.second << endl;
}
```

## Non Standard Problem 1: [Link](#)

State:

$dp[i][x].first$  = is it possible to make a xor of  $x$  from the first  $i$  rows

$dp[i][x].second$  = choice of column that you made

Transition:

$dp[i][x].first$  = true if  $dp[i - 1][x \oplus arr[i][j]].first$  is true for any  $j$  from 1 to  $m$

$dp[i][x].second$  = that particular  $j$  for which  $dp[i - 1][x \oplus arr[i][j]].first$  = true

Base Case:

$dp[0][0].first$  = true,  $dp[0][i].first$  = false

## Non Standard Problem 1: [Link](#)

```
Final Subproblem:
if(dp[n][any value > 0].first is true):
    ans = true
else
    ans = false

Ans construction:
if(dp[n][k].first is true){
    vector<int> choices(n, -1);
    int row = n, xor_value = k;
    while(i != 0){
        int choice = dp[row][xor_value].second;
        int new_xor = xor_value ^ arr[row][choice];
        int new_row = row - 1;
        choices[row - 1] = choice + 1;
        xor = new_xor;
        row = new_row;
    }
    for(auto i : choices)
        cout << i << " ";
}
```

## Representing non-integer parameters

- How will you store the dp states if instead of integer parameters you had a string or a vector or a map or any complex data type?
- Use a map instead of an array.
- Tradeoff `map<pair<int, string>> DP` or `vector<map<string>> DP`

## Non Standard Problem 2: [Link](#) (HW)

- State:
  -
- Transition:
  -
- Base Case:
  -
- Final Subproblem:
  -

## Dynamic Programming Day 5

- Priyansh Agarwal

# Solving Homework Problems

## Problem 1: [Link](#)

```
state:
dp[i][string in last 5 characters] = check if you can
fill question marks in string [i:n] such that there is
no palindrome of length 5 or more in the suffix [i:n]

transition:
dp[i][string in last 5 characters] =
dp[i + 1][string in last 4 characters + '1'] ||
dp[i + 1][string in last 4 characters + '0']

base case:
dp[n][any string so far] = true

final subproblem:
dp[0][""]
```

## Problem 2: [Link](#)

```
state:
dp[i][j] = maximum score that can be obtained from array [
i...j] for the player whose turn it is

transition:
dp[i][j] = max(
    a[i] + sum[i + 1][j] - dp[i + 1][j],
    a[j] + sum[i][j - 1] - dp[i][j - 1])

base case:
    dp[i][i] = a[i]

final subproblem:
    dp[0][n - 1]
```

## Problem 3: [Link](#)

```
if((n * (n + 1)) / 2 is odd)
    ans = 0

state:
ways[value][i] = number of ways to create a sum of value from the
first i elements

transition:
ways[value][i] = ways[value][i - 1] + ways[value - arr[i]][i - 1]

base case:
ways[0][0] = 1, ways[0][anything] = 0

final subproblem:
ways[n * (n + 1) / 4][n]
```



# Cycling DP states

- What happens when your current state is dependent on itself?
  - $dp[i]$  depending on  $dp[i]$  itself

## Problem 1

- Given a positive integer  $N \leq 1e6$ , at every step the following 3 things can happen to  $N$  with equal probability.
  - $N = N / 2$
  - $N = N - 1$
  - $N$  remains unchanged
- Find expected number of steps it will take to convert for  $N$  to become 0

# Problem 1

```
state:
dp[n] = expected number of steps to convert n to 0

transition:

dp[n] = 1/3 (1 + dp[n - 1] + 1 + dp[n / 2] + 1 + dp[n])
dp[n] = 1/3 (3 + dp[n - 1] + dp[n / 2] + dp[n])
dp[n] = 1 + dp[n - 1] / 3 + dp[n / 2] / 3 + dp[n] / 3
2/3 * dp[n] = 1 + dp[n - 1] / 3 + dp[n / 2] / 3
dp[n] = 3 / 2 + (dp[n - 1] + dp[n / 2]) / 2

base case:
dp[0] = 0

final subproblem = dp[n]
```

## Trick to identify a DP problem?

### Repeating subtasks:

- If I have the answer of state, then why should I calculate it again and waste time

### Pro Tips for contests:

- Number of ways problems -> DP, Brute Force or some formula
- Look for small constraints in the problem. (Most probably it would be dp and not greedy)
- Identify states and transition time for each state.
- Calculate time complexity as (number of states \* transition time for each state).
- If this number fits into your Time limit (Great), if not, try to see if you can skip some states and still get the right answer.
- Try to reduce the transition time by using some Data Structure or some clever observation if transition time is the bottleneck
- Never try to over optimize. If your current states and transition time fit into your Time Limit, just code it and do not optimize it further.

# Common states and transitions with constraints

```
total operations <= 1e8  
  
n <= 125 :  
    state:  $O(n^3)$ , transition:  $O(1)$ , [n <= 100  $O(\log n)$  is possible]  
    state:  $O(n^2)$ , transition:  $O(n)$ , [n <= 100  $O(n \log n)$  is possible]  
    state:  $O(n)$ , transition:  $O(n^2)$   
  
n <= 5000:  
    state:  $O(n^2)$ , transition:  $O(1)$  [n <= 1000 then  $O(\log n)$  is possible]  
    state:  $O(n)$ , transition:  $O(n)$  [n <= 1000 then  $O(n \log n)$  is possible]  
  
n <= 1e6:  
    state:  $O(n)$ , transition:  $O(1)$ ,  $O(\log n)$   
  
1 second <= operations <= 4 * 1e8  
4 second <= operations <= 1e9
```

## Dynamic Programming Day 6

- Priyansh Agarwal

# Solving Homework Problems

## Problem 1: [Link](#)

- **State**
  - $dp[i]$  = max profit from  $[0 \dots i]$  in sorted list of projects
- **Transition**
  - $dp[i] = \max(dp[i - 1], \text{profit}[i] + dp[j])$  s.t.  $j$  is the max index for which  $\text{end}[j] < \text{start}[i]$
- **Base Case**
  - $dp[0] = \text{profit}[0]$
- **Final Subproblem**
  - $dp[n - 1]$

# DP with Bitmasking

- Bitmasks
- Basic operations on Bitmasks
- Limitations on “N” (You will need  $2^n$  integers to represent all the subsets)

## Problem 1:

Given a list of points on a 2D plane, rearrange these points in any way such that in the final permutation of points, the sum of distances of the adjacent elements is minimized.

Constraints:  $[N \leq 15]$ ,  $[-1e9 \leq X_i, Y_i \leq 1e9]$

Points :  $\{0, 0\}, \{5, 6\}, \{1, 2\}$

Best permutation  $\rightarrow \{0, 0\}, \{1, 2\}, \{5, 6\}$

Ans =  $\text{Dist}(P1, P3) + \text{Dist}(P3, P2)$

## Problem 1: TC: $O(n^3 2^n)$ , SC: $O(n^2 2^n)$

```
state:
    dp[i][bitmask][last element] = minimum sum of distances in the suffix [i... n - 1]
    such tha bitmask represents the elements in the first i - 1 elements and last elements
    represents the last point

transition:
    check for jth point from (0 to n - 1)
    can you pick the jth point as the ith element in the final array or not

    if(bitmask & (1 << j)){ whether jth bit is set or not
        continue;
    }else{
        dp[i][bitmask][last element] = min(dp[i][bitmask][last element],
        (bitmask != 0 ? dist(j, last element) : 0) + dp[i + 1][bitmask | (1 << j)][j]
    }

base case:
    dp[n][(1 << n) - 1][anything] = 0

final subproblem
    dp[0][0][anything]
```

## Problem 1: TC: $O(n^2 2^n)$ , SC: $O(n * 2^n)$

```
state:
    dp[bitmask][last element]
    i = set_bits(bitmask)
    = minimum sum of distances in the suffix [i... n - 1] such tha bitmask represents the
    elements in the first i - 1 elements and last elements represents the last point

transition:
    check for jth point from (0 to n - 1)
    can you pick the jth point as the ith element in the final array or not

    if(bitmask & (1 << j)){ whether jth bit is set or not
        continue;
    }else{
        dp[bitmask][last element] = min(dp[bitmask][last element],
        (bitmask != 0 ? dist(j, last element) : 0) + dp[bitmask | (1 << j)][j]
    }

base case:
    dp[(1 << n) - 1][anything] = 0

final subproblem
    dp[0][anything]
```

## Problem 2: [Link](#) (Homework)

- State
  -
- Transition
  -
- Base Case
  -
- Final Subproblem
  -